Prompts

1. SETAPESU26/17_wordle help me to complete the task 1 like give me where , what to change give the exact code so that i will paste it there instead of buggy code

2. okay similarly give for task2

3. similarly give for task 3

4. give task4 codes

5. all tasks done?

Below are the ans from the chatgpt

Task 3 — Variable Word Lengths

For Task 3, you need to modify **words.py and main.py**, and make a small change to **game.py**.

The requirement says:

Add 4-, 5-, and 6-letter modes using the supplied in-memory words. The feedback logic must work for all supported lengths.

Currently:

- feedback.py already works with any length because of Task 1.
- game.py accepts a length, but main.py always does WordleGame() → 5 letters.
- words.py currently contains only 5-letter words.

---

1. Change words.py

Open:

17_wordle/words.py

Replace the **entire file** with:

```
WORDS = [
    # 4-letter words
    "area",
```

"bake",

"cold",

"door",

"fish",

"game",

"home",

"lake",

"moon",

"star",

5-letter words

"apple",

"array",

"eerie",

"crane",

"grape",

"stare",

"steam",

"stone",

"plant",

"flame",

6-letter words

"animal",

"basket",

"castle",

"flower",

"garden",

```
    "little",
    "orange",
    "planet",
    "school",
    "summer",
]
```

Now your word list contains:

4 letters → 10 words

5 letters → 10 words

6 letters → 10 words

---

2. Change main.py

Open:

17_wordle/main.py

Currently it is:

```
from game import WordleGame

if __name__ == "__main__":
    WordleGame().run()
```

Replace the **entire file** with:

```
from game import WordleGame



if __name__ == "__main__":
    print("Choose word length:")
    print("4 - Four-letter word")
    print("5 - Five-letter word")
    print("6 - Six-letter word")
```

```
    while True:
        choice = input("Enter 4, 5, or 6: ").strip()

        if choice in {"4", "5", "6"}:
            length = int(choice)
            break

        print("Invalid choice. Enter 4, 5, or 6.")

    WordleGame(length).run()
```

Now when you run:

python main.py

you'll get:

Choose word length:

4 - Four-letter word

5 - Five-letter word

6 - Six-letter word

Enter 4, 5, or 6:

---

3. Change game.py

Your current game.py already has:

```
class WordleGame:
    def __init__(self, length=5):
        self.length = length
        self.target = random.choice([w for w in WORDS if len(w) == length])
        self.history = []
```

This part is actually already designed for variable lengths.

However, I recommend making the validation explicit so only the supported lengths can be used.

Replace the **entire game.py** with:

```
import random
from words import WORDS
from feedback import evaluate



class WordleGame:
    def __init__(self, length=5):
        if length not in {4, 5, 6}:
            raise ValueError("Word length must be 4, 5, or 6.")

        self.length = length
        self.target = random.choice(
            [word for word in WORDS if len(word) == length]
        )
        self.history = []

    def run(self):
        print(f"Wordle — {self.length} letters, 6 guesses.")

        accepted_guesses = 0

        while accepted_guesses < 6:
            guess = input("> ").strip().lower()

            if guess == "q":
```

```
            print("Game quit.")
            return

        if len(guess) != self.length or not guess.isalpha():
            print("Enter a valid word of the required length.")
            continue

        accepted_guesses += 1

        feedback = evaluate(self.target, guess)
        self.history.append((guess, feedback))

        print(" ".join(feedback))

        if guess == self.target:
            print("Solved!")
            return

    print("Out of guesses!")
    print("The word was:", self.target)
```

What you have now

Your project should look like:

```
17_wordle/
|
 ├── README.md
 ├── feedback.py    ← Task 1
 ├── game.py        ← Task 2 + Task 3
```

├── main.py ← Task 3

├── words.py ← Task 3

└── requirements.txt

Task 3 flow

If you select 4:

Choose word length:

4 - Four-letter word

5 - Five-letter word

6 - Six-letter word

Enter 4, 5, or 6: 4

Wordle — 4 letters, 6 guesses.

>

The target is selected only from the 4-letter words.

If you select 5:

Enter 4, 5, or 6: 5

Wordle — 5 letters, 6 guesses.

>

Only 5-letter words are selected.

If you select 6:

Enter 4, 5, or 6: 6

Wordle — 6 letters, 6 guesses.

>

Only 6-letter words are selected.

Important: Task 1 still works

Your feedback.py should remain the corrected version from Task 1:

```
def evaluate(target, guess):
    result = ["gray"] * len(guess)
    remaining = {}

    for ch in target:
        remaining[ch] = remaining.get(ch, 0) + 1

    for i, ch in enumerate(guess):
        if ch == target[i]:
            result[i] = "green"
            remaining[ch] -= 1

    for i, ch in enumerate(guess):
        if result[i] == "green":
            continue

        if remaining.get(ch, 0) > 0:
            result[i] = "yellow"
            remaining[ch] -= 1

    return result
```

Notice that it uses:

```
len(guess)
```

rather than assuming 5.

So the same feedback function works for:

```
target = "home"
guess  = "moon"
```

and:

target = "apple"

guess = "allee"

and:

target = "animal"

guess = "planet"

That is exactly what Task 3 requires.

Task 3 testing

Run:

python main.py

Test these three:

Test A

Enter 4, 5, or 6: 4

Confirm:

Wordle — 4 letters, 6 guesses.

Test B

Restart:

python main.py

Enter:

5

Confirm:

Wordle — 5 letters, 6 guesses.

Test C

Restart:

python main.py

Enter:

6

Confirm:

Wordle — 6 letters, 6 guesses.

Also test an invalid mode:

Enter 4, 5, or 6: 7

Invalid choice. Enter 4, 5, or 6.

Then enter 4, 5, or 6.

One thing to remember

Don't change the Task 1 algorithm while doing Task 3. The progression should be:

Task 1

└── feedback.py

  └── duplicate-letter correction

Task 2

└── game.py

  └── 6 accepted guesses

  └── invalid guesses don't consume turns

  └── win/loss handling

Task 3

├── words.py

| └── 4/5/6-letter words

├── main.py

| └── choose word length

└── game.py

  └── validate supported lengths

This keeps the changes aligned with the repository's existing structure rather than replacing the project with a different implementation.

give task4 codes

Task 4 — Guess History and Final Summary

For Task 4, you mainly need to modify **game.py**.

The requirement is:

- Display readable guess history after **each accepted guess**
- Provide a **final session summary**
- Do **not** add invalid guesses to history
- Keep the existing Task 1, 2, and 3 functionality

You do **not** need to change feedback.py, words.py, or main.py.

Replace the entire game.py

Open:

17_wordle/game.py

Replace everything with:

```
import random
from words import WORDS
from feedback import evaluate



class WordleGame:
    def __init__(self, length=5):
        if length not in {4, 5, 6}:
            raise ValueError("Word length must be 4, 5, or 6.")

        self.length = length
        self.target = random.choice(
            [word for word in WORDS if len(word) == length]
        )
        self.history = []
```

```
def display_history(self):
    print("\nGuess History:")
    for guess, feedback in self.history:
        print(f"{guess}: {' '.join(feedback)}")
    print()

def display_summary(self, won):
    print("\n===== Game Summary =====")
    print(f"Word length: {self.length}")
    print(f"Guesses used: {len(self.history)}")

    if won:
        print("Result: Won")
    else:
        print("Result: Lost")

    print(f"Target word: {self.target}")
    print("========================")

def run(self):
    print(f"Wordle — {self.length} letters, 6 guesses.")

    accepted_guesses = 0

    while accepted_guesses < 6:
        guess = input("> ").strip().lower()

        if guess == "q":
```

```
            print("Game quit.")
            self.display_summary(False)
            return

        if len(guess) != self.length or not guess.isalpha():
            print("Enter a valid word of the required length.")
            continue

        accepted_guesses += 1

        feedback = evaluate(self.target, guess)
        self.history.append((guess, feedback))

        print(" ".join(feedback))

        self.display_history()

        if guess == self.target:
            print("Solved!")
            self.display_summary(True)
            return

    print("Out of guesses!")
    self.display_summary(False)
```

What was added for Task 4

1. display_history()

This function:

```
def display_history(self):
```

```
    print("\nGuess History:")
    for guess, feedback in self.history:
        print(f"{guess}: {' '.join(feedback)}")
    print()
```

prints something like:

Guess History:

crane: gray gray yellow gray green

stare: yellow gray gray green green

It is called after every **valid/accepted** guess:

self.display_history()

---

2. display_summary()

This gives the final session summary:

===== Game Summary =====

Word length: 5

Guesses used: 3

Result: Won

Target word: apple

==========

For a loss:

===== Game Summary =====

Word length: 5

Guesses used: 6

Result: Lost

Target word: apple

==========

---

3. Invalid guesses are NOT recorded

This is important for Task 4.

The code does:

if len(guess) != self.length or not guess.isalpha():

    print("Enter a valid word of the required length.")

    continue

before:

accepted_guesses += 1

self.history.append((guess, feedback))

Therefore:

> abc

Enter a valid word of the required length.

doesn't appear in the history.

Your final project structure

After Task 4:

17_wordle/

│

├── README.md

├── feedback.py

├── game.py

├── main.py

├── words.py

└── requirements.txt

feedback.py

Task 1 — duplicate-letter feedback.

game.py

Tasks 2 + 4, plus variable-length support from Task 3.

main.py

Task 3 — 4/5/6-letter selection.

words.py

Task 3 — 4-, 5-, and 6-letter words.

Test Task 4

Run:

python main.py

Choose:

5

Then make a few valid guesses.

After every guess you should see:

Guess History:

guess1: ...

guess2: ...

guess3: ...

Then test an invalid guess:

> abc

Enter a valid word of the required length.

It should **not** appear in the history.

Finally test:

Winning

When you enter the target:

Solved!

===== Game Summary =====

Word length: 5

Guesses used: X

Result: Won

Target word: XXXXX

==========

Losing

After six valid incorrect guesses:

Out of guesses!

===== Game Summary =====

Word length: 5

Guesses used: 6

Result: Lost

Target word: XXXXX

==========

Quitting

Entering:

q

produces:

Game quit.

===== Game Summary =====

...

Result: Lost

...

So after this, **Tasks 1–4 are implemented** according to the README requirements.

Today 4:35 PM

all tasks done?

Yes. Based on the repository's README and the code changes we've made, **Tasks 1–4 are covered**.

Task	Status	Files
Task 1 — Duplicate-letter feedback	✅	feedback.py
Task 2 — Win/loss + 6 accepted guesses + invalid guesses don't consume turns	✅	game.py
Task 3 — 4/5/6-letter modes	✅	main.py, words.py, game.py
Task 4 — Guess history + final summary	✅	game.py

Before submitting, test these required cases

The README specifically asks for:

- ✅ Repeated-letter cases
- ✅ Exact matches
- ✅ Absent letters
- ✅ Invalid lengths
- ✅ 4-letter mode
- ✅ 5-letter mode
- ✅ 6-letter mode
- ✅ Winning on the **6th** allowed guess
- ✅ Losing after 6 guesses
- ✅ Quitting
- ✅ Invalid attempts not appearing in history

One important point: **don't submit yet just because the code is changed.** Run the tests above and make sure the behavior actually works.

Also, the repository's submission checklist requires:

1. **10-second video before changes** showing the bug
2. **10-second video after changes** showing the fix/features
3. **Chat/LLM page link with the complete chat history**

If you want, I can now give you a **complete Task 1–4 testing sequence**, including exactly what to type in the terminal so you can verify everything before submitting.